Mizar Evo の設計指針について

自動証明・数学的記述・検証可能な計算の再接続

中正 和久

岩手県立大学

TPP 2026

理化学研究所 AIP 東京オフィス

2026 年 11 月 16 日〜17 日

0.2 - Mizar の実装史: 初期処理系から Mizar-rs、Evo へ

表 1　Mizar の初期処理系と MML の成立

時期	実装史の主な節目
1973 – 75	数学文書の構想 → 命題論理の Mizar-PC（Algol / ODRA）
1977	Mizar-QC: 量化を導入（Pascal / CDC 6000）
1982 – 84	MSE: 小言語への整理。HPF: 数学的記法・暗黙の引数を拡充
1986	Mizar-4: 複数パスで処理。同年末に PC / DOS へ移植
1988 – 89	PC-Mizar、article 間参照。1989 年 1 月 1 日に MML 開始

表 2　PC-Mizar・Mizar-rs・Mizar Evo の概要

処理系	概要
PC-Mizar（Pascal）	Parser → MSM → Analyzer → Checker の段階的処理
Mizar-rs（Rust、ITP 2023）	既存言語との互換性を目指し checker / analyzer を再実装 全 MML 検査: checker を差し替え、当時の全 MML を検査 高速化: 約 5〜6 倍（8 threads、実時間比較） 不具合発見: 既存実装の健全性バグ 4 件を報告（MML 内で偽の証明への利用なし）
Mizar Evo（Rust）	数学的記述と論理基盤を継承し、新しい言語仕様・処理系を構築

Source: Matuszewski & Rudnicki (2005); Carneiro (ITP 2023), Table 1。

0.3 - 2023 – 2026 年: 構想から言語設計へ

表 3　Mizar Evo の開発経緯

時期	開発内容
2023 年 8 月	初回 README は mizar-wishlist
2024 年 9 – 10 月	import、namespace、structure 継承、型システム
2025 年 5 – 6 月	mode、型推論、redefinition。依存構造・属性の草案を削除後、再導入
2026 年 1 – 4 月	EBNF・論理エンコード、registration、template、algorithm の意味論・契約
2026 年 5 月	実装設計 → テストハーネス → minimal lexer

- **Mizar の記法を拡張する構想から出発し、2023 年の科研費申請準備を機に、言語仕様と処理系を再設計する方針を定めた。**
- ITP 2023 で Mario Carneiro 氏に実装協力を相談し、前向きな了承。CICM 2023 では Adam Naumowicz 先生に wishlist について相談。
- **当初の柱**: 型の全称量化（generics）、namespace / package、ATP / traceability。Mizar-rs を利用するプロトタイプを構想。

科研費 基盤研究 (C)・2024 年度採択: 24K14897「集合論ベースの定理証明支援系の言語仕様の刷新」

0.4 - 設計の深化と AI Agent による実装

言語設計の深化

- **template:** 当初の generics 構想から、scheme 相当を theorem に統一。構造 view・型引数推論へ拡張。
- **algorithm:** 日本の Mizar コミュニティの暗号理論研究者から、当初より アルゴリズムを証明したいという要望。楕円曲線の結合法則の証明でも、膨大な数式処理に苦労。

2026 年 5 月からの実装と意見交換

- **5 月: AI coding agents を使い Rust 実装を本格化。6 – 7 月: crate 開発・自律開発手順・SAT-backed 検査と監査。**
- **設計 → エージェント実装 → テスト・監査 → 仕様再検討。**
- 9 月: Białystok で Mizar チームにセミナー・意見交換。
- 9 – 10 月: source 由来の VC を拡張しつつ、algorithm の部分性・契約を修正。

0.5 - 現行 Mizar の課題と 6 つの設計指針

表 4　現行 Mizar の課題と 6 つの設計指針

領域	現行 Mizar から取り組む課題	新仕様の設計指針
§ 1 基盤	論理・MML の継承と処理系の刷新	一階論理・集合論を維持し、kernel を小さく保つ
§ 2 記述	暗黙の type・registration・overload の追跡	数学的な抽象化を継承し、暗黙の選択を明示
§ 3 汎用化	定義・定理・scheme の個別機構	template で共通化の仕組みを統一
§ 4 計算	証明と実行可能な手続きの接続	algorithm の契約・不変条件・停止性を検査
§ 5 開発基盤	依存管理・配布・差分検証・IDE	module・namespace・package・差分ビルド・LSP
§ 6 検査・自動化	外部探索と検査根拠の接続	ATP を活用し、evidence のインスタンス化と SAT 検査

1.1 - 基盤の継承: 論理は保ち、処理系を再整備

数学的記述の層を継承し、
処理系と開発基盤を再構築する。

- **一階論理と Tarski-Grothendieck 集合論を基盤として維持。**
- soft type、mode、attribute、registration、structure、**宣言的証明を継承。**
- **ATP の探索と検査を分離し、結果を小規模な kernel で独立に検査。詳細は § 6。**

1.2 - 集合から位相空間まで

例 1: Cantor の定理（現行 Mizar・証明省略） sketch

```
theorem Cantor:
  for X being set holds card X in card (bool X)
proof ... end;
```

例 2: コンパクト空間から Hausdorff 空間への連続全単射（証明省略） sketch

```
theorem CompactBijection:
  for S being compact TopSpace,
      T being non empty T_2 TopSpace,
      f being Function of S,T
  st f is continuous one-to-one & rng f = [#] T
  holds f is being_homeomorphism
proof ... end;
```

- **集合・冪集合・基数から位相空間まで、一階の集合論の上で数学を記述する。**

2.1 - type • registration • overload の追跡

数学を自然に記述するため、処理系は多くの判断を自動的に行う。

- **type: 対象をどの型として使えるか。** 属性や継承をたどる。例: Group → Magma（継承経路が一意の場合）。
- **registration: 必要な属性は、どの規則から得られたか。** 例: empty → finite → countable という登録規則の連鎖。
- **overload: どの定義を選び、どのビューで使うか。** 同じ記号の候補や、環の加法・乗法のビューを区別する。

自然な記述と自動適用を保ちながら、暗黙の判断の根拠を追跡可能にする。

2.2 - type: field と property を宣言する

構造の宣言（definition 内の抜粋）

specification example

```
struct LoopStr where
  field carrier -> set;
  field binop -> BinOp of carrier;
  property unit -> Element of carrier;
end;

struct AddLoopStr where
  field carrier -> set;
  field add -> BinOp of carrier;
  property zero -> Element of carrier;
end;
```

field は格納するデータ。property は名前と型を宣言し、実装は別に与える。

2.3 - type: 継承関係を後から宣言する

AddLoopStr の宣言後に、親への対応付けを追加 specification example

```
definition
  inherit AddLoopStr extends LoopStr where
    field carrier from carrier;
    field add from binop;
    property zero from unit;
  end;
end;
```

- **構造の宣言と継承関係を分離。型を定義した後で Rust の trait を実装する構成に近い。**
- **`from` で親の役割をリネーム**: `binop` を `add`、`unit` を `zero` に対応付ける。
- **親ごとに一つの宣言。同じメンバー型なら証明不要、型を狭める場合は `coherence` を証明する。**

2.4 - type: Unital の unit を実装し、zero に引き継ぐ

has　unit は両側単位元の存在を表す（unit に依存しない） sketch

```
definition
  mode UnitalDef: Unital is has_unit LoopStr;
  let M be Unital;
  property M.unit means
    for x being Element of M.carrier holds
      M.binop(it,x) = x & M.binop(x,it) = x;
  existence proof ... end;
  uniqueness proof ... end;
end;
```

- A の LoopStr ビューが Unital なら、unit の実装を zero として利用できる。
- `property zero from unit` の対応: `A.zero = (A qua LoopStr).unit`。zero の再実装は不要。

2.5 - registration: 自動連鎖で見えにくくなる依存

自動適用が連鎖し、利用箇所のソースだけでは依存する登録規則を追いにくい。

```
registration
  cluster EmptyImpliesFinite: empty -> finite for set;
  coherence proof ... end;
  cluster FiniteImpliesCountable: finite -> countable for set;
  coherence proof ... end;
end;
```

表 5　新仕様で記録する登録規則の適用経路

出発点	自動的に得られる事実	記録する規則
S is empty	S is finite	EmptyImpliesFinite
S is finite	S is countable	FiniteImpliesCountable

- `empty` **の宣言から** `finite`・`countable` **まで暗黙に得る。**`by` **で規則を引用する必要はない。**
- **新仕様: 自動適用を保ち、規則のラベルと適用経路を記録する。**

2.6 - overload: 同じ環の単位元を使い分ける

同じ環 R でも、加法の単位元は 0、乗法の単位元は 1。

- **加法の** AddMagma: `0 + x = x + 0 = x`
- **乗法の** MulMagma: `1 * x = x * 1 = x`

id. M は unital Magma M の単位元を返す sketch **(定義省略)**

```
let R be Ring;
id. R;                        :: ambiguous Magma view
id. (R qua AddMagma);          :: R.zero
id. (R qua MulMagma);          :: R.one
```

- id. **の定義が一つでも、**R **をどの** Magma **として使うかが曖昧になる。**
- **qua で加法・乗法のビューを明示し、0 と 1 を区別する。**

3.1 - 現行 MML: 型ごとに増える再定義・登録

関数の和の本体は一つでも、結果型を保つ記述は型ごとに必要。

表 6　関数の和に付随する型ごとの再定義・登録（VALUED_1）

値の型	結果型の再定義	属性・全域性の登録
複素数	`PartFunc of C,COMPLEX`	`complex-valued + total`
実数	`PartFunc of C,REAL`	`real-valued + total`
有理数	`PartFunc of C,RAT`	`RAT-valued + total`
整数	`PartFunc of C,INT`	`INT-valued + total`
自然数	`PartFunc of C,NAT`	`natural-valued + total`

- **対応する型が増えるたびに、再定義・登録・整合性の証明が増える。**
- **同じ「関数の和」と、その結果型を、一つの** template **で記述したい。**

3.2 - template: 本体と結果型を一元化する

汎用 functor sketch (存在・一意性の証明は省略)

```
definition
  let T be type extends non empty AddMagma;
  let I be non empty set;
  let f, g be Function of I, T.carrier;
  func AddDef: Add[T,I](f,g) -> Function of I,T.carrier means
    for i being Element of I holds it.i = T.add(f.i,g.i);
  synonym f +[T,I] g for Add[T,I](f,g);
end;
```

表 7　Template の具体化と結果型

呼び出し（実数 R・複素数 C）[1]	結果型
f + g	Function of I,R.carrier
u + v	Function of I,C.carrier

- 有理数・整数・自然数にも、同じ Add[T,I]。本体と結果型の記述を一元化する。

[1]明示形: f +[R,I] g、u +[C,I] v。省略は宣言型から一意に推論できる場合。継承経路が曖昧なら qua を明示。

述語パラメータ付き定理としての帰納法 specification example

```
definition
  let P be pred(Nat);
  theorem NatInduction[P]:
    P(0) & (for n being Nat st P(n) holds P(n+1))
    implies for n being Nat holds P(n)
  proof ... end;
end;
```

- **述語パラメータにより、代入する述語ごとの一階定理の族を表現する。**[1]
- **明示的なインスタンス化**: `defpred` **に続けて** `by NatInduction[P], Base, Step`**。**
- **関数子パラメータは集合ではなく** schema **レベルの記号として扱い、一階論理を維持する。**

[1] **詳細**: Białystok **資料** Story 6（`PermProduct[T]`、`qua` **によるビュー**）、**および本資料** Backup 1**。**

3.4 - template: 構造と属性もパラメータ化する

struct: 体 K 上のベクトル空間のデータ sketch (definition 内の抜粋)

```
let K be Field;
struct VectorSpace[K] where
  field carrier -> non empty set;
  field add -> BinOp of carrier;
  field smul -> Function of [:K.carrier, carrier:], carrier;
end;
```

of / over は syntax sugar: Vector of K → Vector[K]（mode の例）。

attr: 空間 M における部分集合 S の稠密性 sketch (条件省略)

```
let M be MetricSpace;
let S be Subset of M.carrier;
attr DenseDef: S is dense[M] means ...;
```

属性の接頭表記も syntax sugar: M-dense → dense[M]。

型・値によるパラメータ化は struct・attr・mode・pred・func に共通。

4.1 - 計算の課題: 検証可能な algorithm

手続きを Hoare 論理の規則で証明義務に分解する:

$$\{\ \mathit{requires}\ \}\ \mathit{algorithm\ body}\ \{\ \mathit{ensures}\ \}$$

P・Q: **事前・事後条件**、B: **条件**、I: **不変条件**、S: **文**。

表 8　Hoare 論理の基本規則

構文	検査する条件（部分正当性）[1]
代入 `x := e`	`P ⇒ Q[x ← e]`（Q **中の自由な** x **を** e **で置換**）
`if B do S1 else S2 end`	`{P ∧ B} S1 {Q}` **と** `{P ∧ ¬ B} S2 {Q}`
`while B do S end`	**初期成立**: `P ⇒ I`、**保存**: `{I ∧ B} S {I}`、**終了**: `I ∧ ¬ B ⇒ Q`
`return e`	`P ⇒ Q[result ← e]`

各構文の規則を再帰的に適用し、一階論理の証明義務へ分解する。

[1] **単純変数・純粋な式の基本形。途中の** return / break / continue**、型・定義域の条件は省略。**

契約付きのユークリッドの互除法 specification example

```
terminating algorithm EuclidGcdDef: euclid_gcd(a, b) -> Nat
  requires a >= 1 & b >= 1
  ensures result = Gcd(a, b)
do
  var x := a;  var y := b;
  while y <> 0 do
    invariant x >= 1 & y >= 0 & Gcd(a, b) = Gcd(x, y);  decreasing y;
    const r := x mod y;  x := y;  y := r;
  end;
  return x;
end;
```

- `terminating` は requires を満たすすべての入力での停止を要求。不変条件と `y` の厳密な減少を検査する。[1]
- 検証後は数学的な functor に昇格し、論理式・証明の中で使える。

[1] ループ注釈は Dafny の `invariant` / `decreases` に類似。Evo は `invariant` / `decreasing` を使う。

互除法の本体を再掲（a,b >= 1、外枠省略） sketch

```
var x := a; var y := b; :: (1)
while y <> 0 do
  invariant x >= 1 & y >= 0 & Gcd(a,b) = Gcd(x,y); :: (2)
  decreasing y; :: (3)
  const r := x mod y; x := y; y := r;
end;
return x; :: (4)
```

表 9　ユークリッドの互除法の証明義務

箇所	証明義務	数学的な根拠
(1)	不変条件の初期成立	x=a, y=b。事前条件から正値性
(2)	不変条件の保存	旧 (x,y) から (y,r) へ。Gcd(x,y) = Gcd(y,r)、旧 y >= 1、r >= 0
(3)	Nat 値の測度の減少	y' = r、0 <= r < y
(4)	終了時の事後条件	y=0。Gcd(a,b) = Gcd(x,0) = x

各箇所をライブラリの GCD・剰余の補題とともに検査する。

4.4 - terminating と functor 昇格

表 10　Algorithm の停止性と保証

algorithm の形	requires の下での意味
terminating なし	呼出しが戻れば契約が成立
terminating を検証済み	requires の下で全域性。functor として利用

互除法は契約を通じて推論する sketch

```
let a, b be Nat;
assume a >= 1 & b >= 1;
thus euclid_gcd(a,b) = Gcd(a,b) by EuclidGcdDef;
```

計算結果を使う証明 sketch (MVM 未実装)

```
thus euclid_gcd(12,8) = 4 by computation;
```

検証済み algorithm **を** MVM **（仮想機械）で実行し、その結果を証明に使う。**
既定は MVM も信頼する `computed` **経路。kernel/SAT の再検査とは区別する。**

4.5 - 式変形を検証する: lambda と微分

関数を直接記述する（u は Real） sketch

```
u |-> (sin u)*u
```

differentiate が積・sin などの式の構造を再帰的に処理する。

微分の証明に利用する（x は Real） sketch (MVM 未実装)

```
thus diff((u |-> (sin u)*u), x) = (cos x)*x + sin x
  by computation(differentiate(u |-> (sin u)*u));
```

- |-> は集合論的な関数を表し、expr（構文木）が式の構造と変数束縛を保持する。
- 変換 algorithm の契約で、微分可能性と導関数の正しさを検証する。[1]

[1] 微分則・意味変換・簡約の補題を仮定。仕様例であり、実行結果ではない。

5.1 - 環境部: 著者は何を取り込むのか

現行 Mizar の環境部、ALGSTR_0 を短縮 sketch

```
environ
 vocabularies ... STRUCT_0 ...;
 notations ... STRUCT_0;
 constructors ... STRUCT_0 ...;
 registrations ... STRUCT_0;
 theorems STRUCT_0;
```

表 11　環境部の 10 種類の指令

一覧	取り込むもの
vocabularies / notations	記号 / 記法
constructors / registrations	構成子 / 自動的な型の事実
requirements / definitions	特別な自動処理 / 定義情報
equalities / expansions	等式 / 定義展開の情報
theorems / schemes	定理 / scheme

- notations・definitions: article 順が記法・定義の優先を決める。
- **アーティクル執筆の最大の難所: 環境部の上書きと順序依存の管理。**

5.2 - import と依存の追跡

新仕様: モジュールの import specification example

```
import .function;
import mml.algebra.structure.sorted;
```

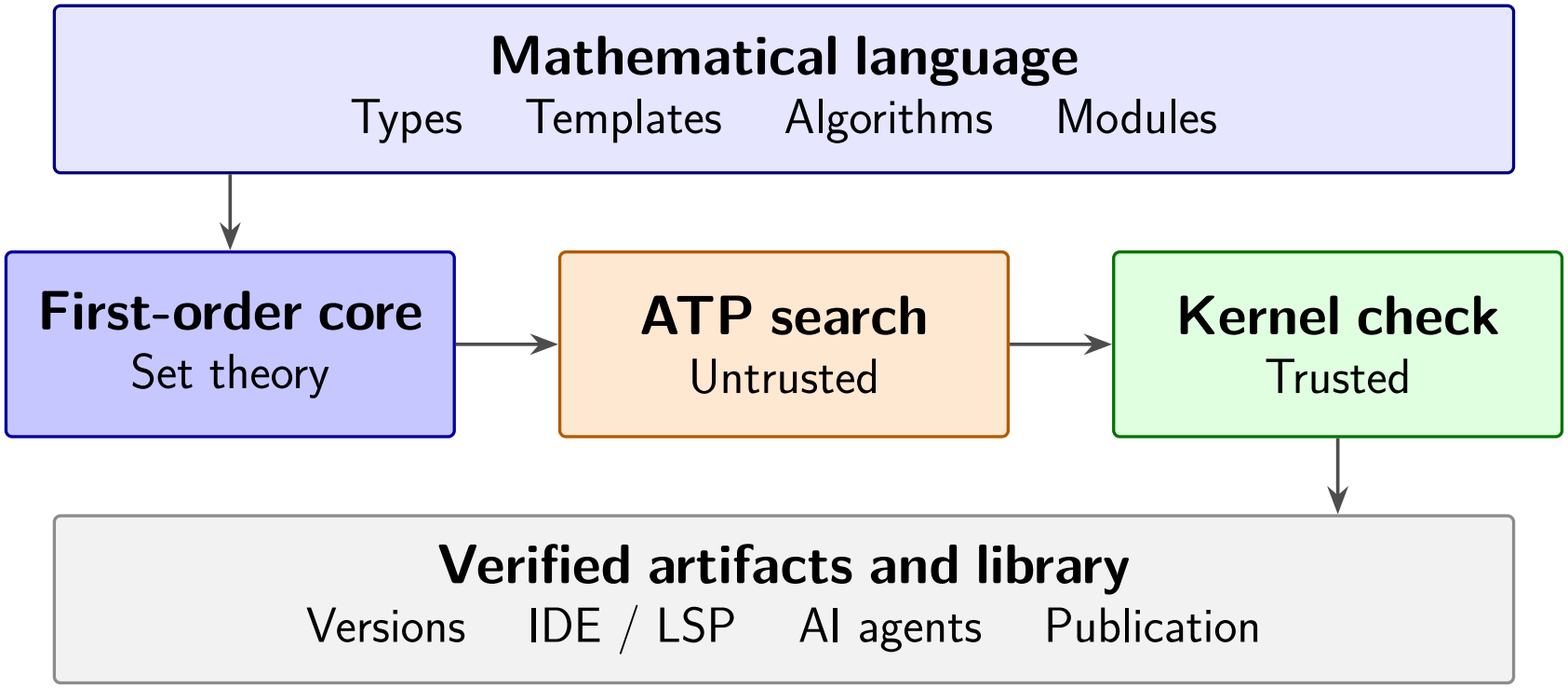
表 12　import と依存関係の追跡

段階	著者が確認できるもの
import	モジュールの公開された定義・定理・登録
解決	元の完全修飾名と、登録の解決トレース
再利用	差分ビルドで検証済みの依存指紋

- **公開項目をまとめて取り込み、実際に何を使ったかを追跡する。**
- package・lock file **で依存バージョンを固定し、**IDE **診断で解決した名前・トレースを確認する設計。**

5.3 - 全体像

deep dive



- **豊かな数学的記述を、小規模な一階論理の基盤と現代的な開発環境で支える。**

6.1 - 自動化: 補題を探索し、証明を提案する

Isabelle/HOL: Sledgehammer が証明を提案 sketch

```
have "Q a"
  sledgehammer
  by (metis allPQ pa)
```

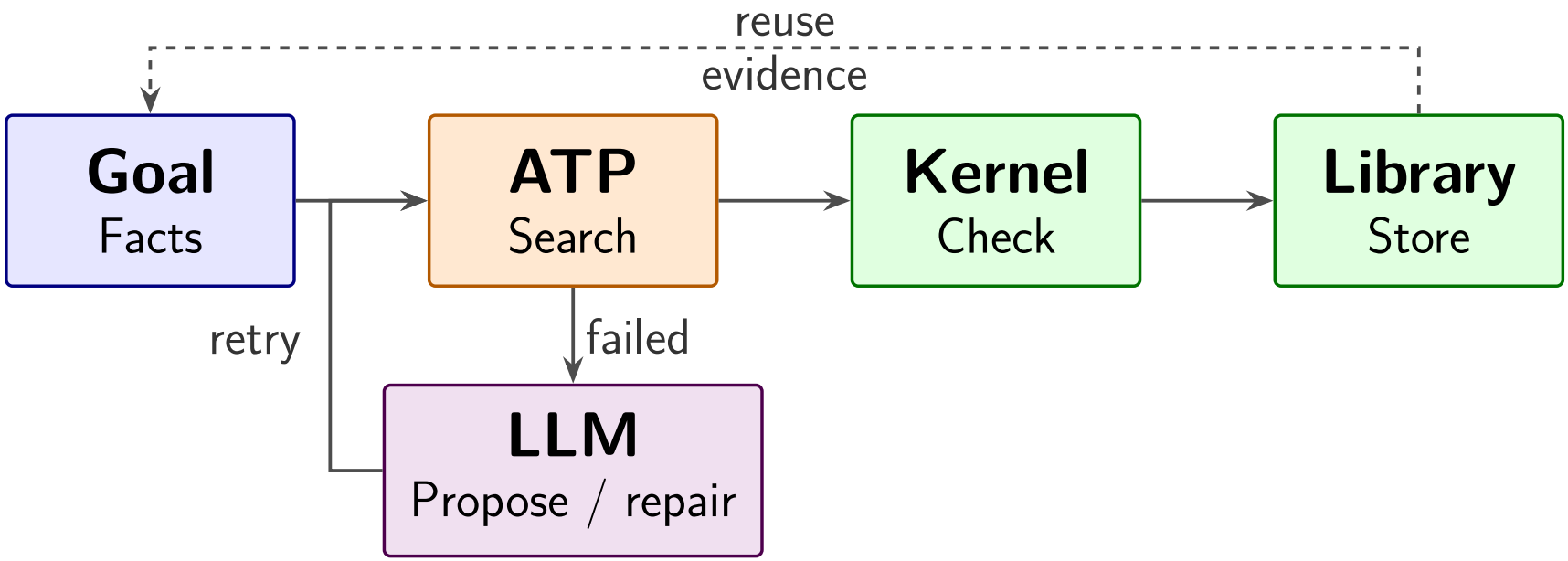
Mizar Evo: by atp で補題探索を依頼（暫定構文）。
未解決ゴール → ライブラリの補題候補 → ATP 試行 → by の提案。

入力と提案例（Pa は局所仮定 P(a)） sketch (予定)

```
thus Q(a) by atp;
:: 提案例:
:: library.AllPQ: for x holds P(x) implies Q(x)
thus Q(a) by library.AllPQ, Pa;
```

- **補題を探して列挙する負担を軽減し、採用した前提はソースに明示する。**
- **提案を反映して再検証し、論理式・置換の** evidence **を** kernel / SAT **で検査して受理する。**

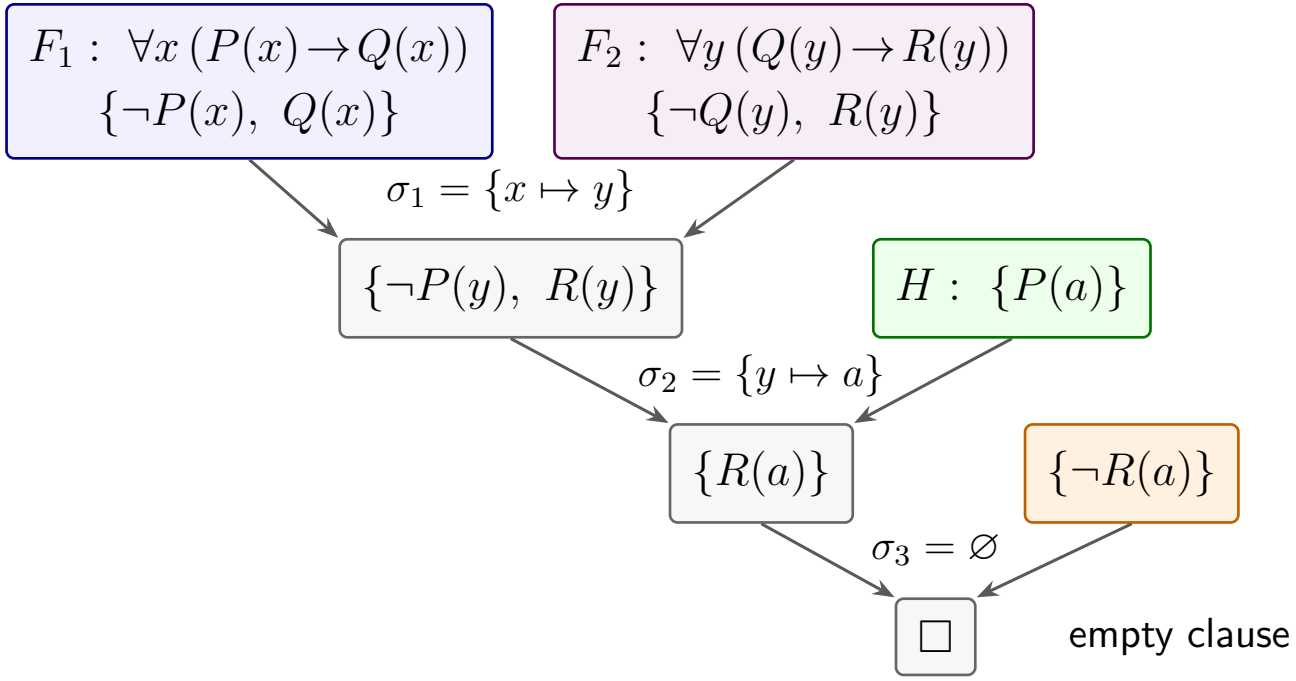
6.2 - ATP の活用: 探索・検査・再利用



- **ゴールと前提 → ATP 探索 → kernel 検査 → ライブラリへ保存・再利用。**
- **未解決時は補題・方針を修正して再探索。図は設計意図。**

Resolution ログからの候補抽出案 sketch

Untrusted resolution log (sketch)



Extracted evidence

Compose substitutions along each source branch:

$$F_1 : \ x \mapsto y \mapsto a$$
$$F_2 : \ y \mapsto a$$
$$\Longrightarrow \ F_1[x := a]$$
$$\Longrightarrow \ F_2[y := a]$$

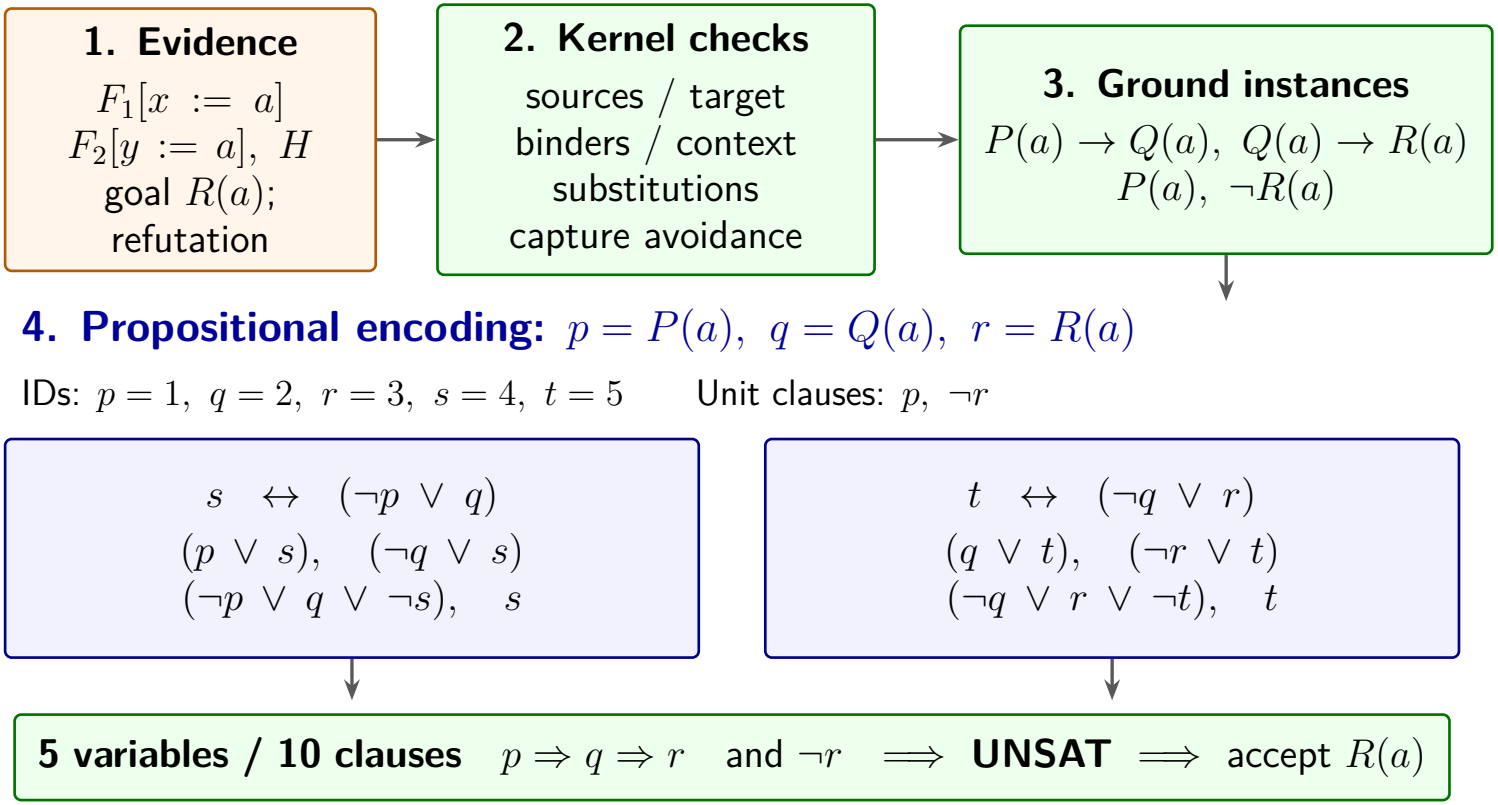
Retain F_1, F_2, H, bindings, context and goal $R(a)$.

The kernel checks evidence; it does not replay this tree.

- **非信頼の抽出器が枝上の置換を合成。F1 に `x := a`、F2 に `y := a` を保存する。**

6.4 - 保存する証拠から具体的な SAT 節へ

evidence の照合から SAT による受理まで sketch



- evidence を照合し、CNF を再生成。SAT で UNSAT を確認して R(a) を受理。

7.1 - 実装状況と規模

Rust 製品コード: 実装済み 271.1 kLoC、完成時見積 360 – 540 kLoC（暫定）。

表 13　実装状況とコード規模（kLoC）

領域	実装済み	完成時見積	限定実装と残る範囲
lexer / parser / frontend	34.4	36 – 43	syntax を含む。実ソース読込・字句／構文解析を接続
resolver / checker	122.9	148 – 193	source 由来の限定意味論。一般化・サービス接続は残る
Core / VC	49.7	65 – 95	IR・proof を含む。限定 source から検証義務へ
ATP	9.6	16 – 28	問題符号化・候補生成。実 backend との統合は残る
kernel	12.8	15 – 20	式・置換・来歴検査。SAT 符号化・接続 1.9 を含む
build / driver / artifact	41.3	56 – 85	cache・共通基盤を含む。全工程の発行は未接続
LSP	0.5	6 – 19	lifecycle・文書同期の基盤。意味情報との接続は残る
MVM / extraction / doc	0	17 – 57	runtime・抽出・文書生成は未実装

ATP を除く検査部: kernel と SAT 依存を合わせて 17.6 kLoC。

Core・session 等の共通基盤は別計上。表は依存・テスト・空行・コメントを除外。見積は現実装を含み、各値は丸め。

Source: 固定版 08861708。集計・見積根拠: `draft.ja.md` § 7。

7.2 - 今後の開発とまとめ

今後の開発

- **最優先: source-to-semantics → VC → evidence → kernel → artifact → build の統合。**
- Algorithm **の意味論・検証義務を整合し、限定的に動く経路を広げる。**
- MVM runtime**・抽出は別途開発。**MML **全体の移植は処理系完成と別課題。**
- **学習的前提選択、**LLM **修復、暗号・量子は長期構想。**

まとめ

- **FOL・集合論と数学的な記述能力を継承し、言語・開発基盤を再設計する。**
- **ATP の探索と信頼できる検査を分ける。仕様は今も進化中。**
- **ITP をお使いの皆様から、記述・証明支援・開発環境への要望を伺いたい。**

Backup 1. template のインスタンス化: 一つの証明、多くのビュー

有界な型パラメータと汎用的な定理 specification example (証明は省略)

```
definition
  let T be type extends commutative associative unital Magma;
  theorem PermProduct[T]:
    for s being FinSequence of T,
        p being Permutation of dom s
    holds Product[T](s) = Product[T](s * p)
  proof ... end;
end;
```

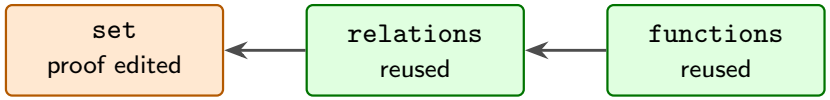
インスタンス化 specification example (必要な registration がある前提)

```
PermProduct[commutative associative unital AddMagma]
PermProduct[commutative associative unital MulMagma]
let R be commutative Ring;  PermProduct[R qua AddMagma]   :: additive view
```

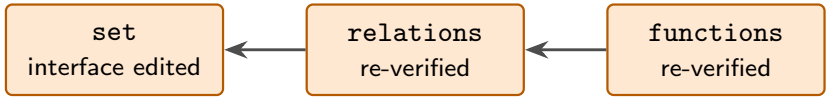
- 環から Magma への継承経路は二つ。qua によるビューの選択が、記法を決定する。

Backup 2. 差分検証

theorem proof edit:
public statement + accepted status unchanged — importers reuse



interface change:
the dependency cone re-verifies



reuse requires all relevant keys: source/dependency slices, package/lockfile, toolchain/schema, registration/cluster, verifier policy, evidence/witnesses; missing data is a cache miss — a clean build must reproduce every acceptance

- **証明本体の編集時、公開された主張と受理状態が不変なら、依存側の再ビルドは不要。インタフェース変更時は依存コーンを再検証。**
- **キャッシュ再利用には、関連キーの完全一致が必要。データ欠落時はキャッシュミス。再利用自体は証明の正当性を保証せず、クリーンビルドで全受理結果を再現する。**

Backup 3. Package manifest

Mizar Evo specification example

```
[package]
name    = "algebra"
version = "2.3.1"
edition = "2025"

[dependencies]
mml_core = "^1.0"
topology = { version = "^0.9", features = ["metric"] }
```

- **再現可能なビルドの要件**: **固定したソース・**lockfile**・ツールチェーン・検査器設定・決定的な** ATP evidence**。**
- **バージョン管理された再利用により、**article **集合間の手作業コピーを解消。集約モジュールで、一分野を一つの** import **として公開できる。**

Source: doc/spec/en/23.package_management_and_build_system.md; Białystok **資料** Story 1**。**

Backup 4. 評価条件の比較

表 14　MizAR 60 と Sledgehammer の評価条件

	MizAR 60 (ITP 2023)	Sledgehammer / AFP (ITP 2022)
報告された成功率	58.4%（1,690 / 2,896）	68.8%（3,440 / 5,000）
評価対象	MML 1147 の定理・補題。全 57,897 件のうち holdout 2,896 件	AFP の 50 entry から選んだ局所ゴール 5,000 件
前提選択	ライブラリ全体から学習的に選択	MePo。基準 512 事実、構成により増減
時間予算	ポートフォリオ合計 420 CPU 秒	greedy 16 構成 × 各 30 CPU 秒 = 480 CPU 秒
成功の判定	ATP 証明の発見（hammering mode）	外部 prover の証明発見。Isabelle 内の再構成は評価対象外

- **評価対象・前提選択・時間予算・構成選択が異なる。成功率だけでは設計上の優劣を判定できない。**

Backup 5. HOL の符号化と再構成

符号化の手順の概形 sketch

```
F X                    ->  app(F, X)
(%x. t) ...            ->  fresh constant + defining axioms   (lambda lifting)
polymorphic types      ->  type guards or type tags
Boolean-valued terms   ->  extra encoding
```

- 関数変数の適用は app で明示。ラムダ抽象は lambda lifting またはコンビネータで、型は guard または tag で符号化する。
- ATP の証明を Isabelle 内で再構成（reconstruction）：metis、smt の replay、生成した Isar テキストを利用。
- 高階 ITP と一階 ATP の接続を実現する技術。
- 接続の仕組みを示す説明であり、実測上の性能の優劣を示すものではない。